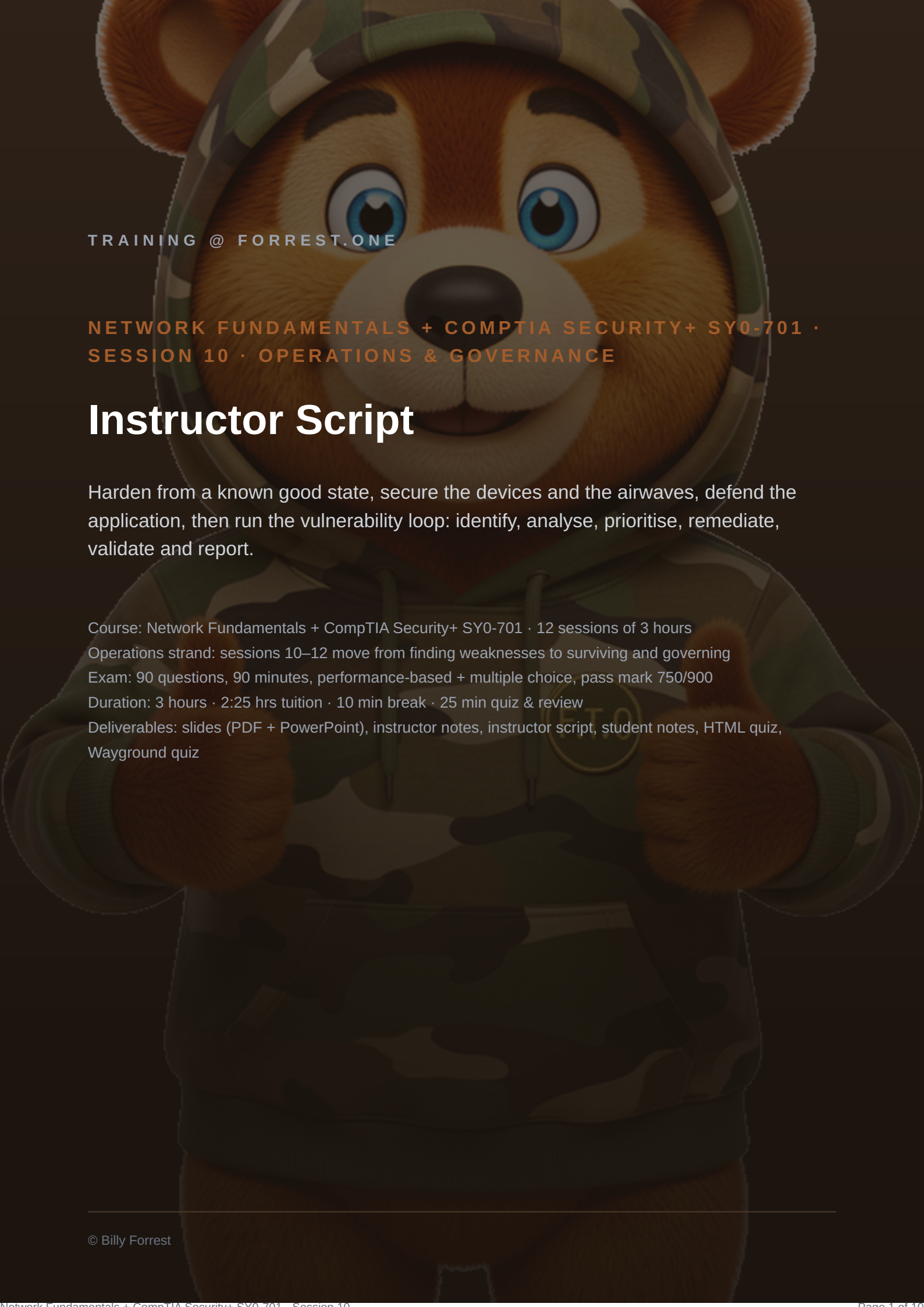




TRAINING @ FORREST.ONE

NETWORK FUNDAMENTALS + COMPTIA SECURITY+ SY0-701 ·  
SESSION 10 · OPERATIONS & GOVERNANCE

# Instructor Script

Harden from a known good state, secure the devices and the airwaves, defend the application, then run the vulnerability loop: identify, analyse, prioritise, remediate, validate and report.

Course: Network Fundamentals + CompTIA Security+ SY0-701 · 12 sessions of 3 hours

Operations strand: sessions 10–12 move from finding weaknesses to surviving and governing

Exam: 90 questions, 90 minutes, performance-based + multiple choice, pass mark 750/900

Duration: 3 hours · 2:25 hrs tuition · 10 min break · 25 min quiz & review

Deliverables: slides (PDF + PowerPoint), instructor notes, instructor script, student notes, HTML quiz, Wayground quiz

## Instructor Script: Vulnerability Management, Resiliency & Asset Protection

A word-for-word delivery script for Session 10, following the expanded narration chapter by chapter. Stage directions appear in square brackets. Each slide runs roughly 45 to 90 seconds of speech; the activities and discussion expand the timings in the instructor notes.

### Slide 1 · Title

Welcome to Session 10, Vulnerability Management, Resiliency and Asset Protection. Today we do the work that keeps a network defensible over time. First we build systems from a known secure state, then we hold them there. Then we hunt vulnerabilities, judge them by real risk, and fix them in a managed order. Along the way we secure mobile devices, wireless networks and applications, and we account for every asset from purchase to destruction.

### Slide 2 · Six chapters, from a secure baseline to a tuned alert

Here is the shape of the session. Chapter one: baselines and hardening, the known good state and the tools that hold it. Chapter two: mobile devices and wireless, the equipment that leaves the building and the air inside it. Chapter three: applications and monitoring, because that is where most attacks land and where the evidence appears. Chapter four: the asset lifecycle, because you cannot protect what you do not know you own. Chapter five: the vulnerability loop, the heart of the session. And chapter six: the monitoring tooling that turns raw events into decisions. One thread runs through all six: you cannot protect what you have not hardened, and you cannot fix what you cannot find.

### Slide 3 · Session objectives

By the end of these three hours you will be able to implement a baseline and detect drift, harden the operating system, the application and the firmware, secure mobile devices and wireless networks, defend an application, monitor the estate, manage an asset from acquisition to destruction, run the vulnerability loop, and tune the tooling so the right alert reaches you in time. The quiz samples every one of them.

### Slide 4 · Chapter one: baselines and hardening

Every system should start from a known secure state. A secure baseline is the documented minimum configuration a system needs before deployment, and any deviation from it is configuration drift. Without one, two machines that should be identical drift apart and nobody can say what normal looks like. This chapter covers the baseline, the tools that enforce it, and hardening at three layers.

## Slide 5 · A known secure state

---

A secure baseline is the documented set of minimum security settings a system needs before it is deployed. Drift arrives through unauthorised changes and missed updates, and it is quiet, which is what makes it dangerous. The second benefit is verification: with a written standard, an auditor compares the machine against the baseline rather than arguing an opinion. Start secure, then prove you stayed secure.

## Slide 6 · Implementing a baseline

---

Implementing a baseline is a small project with four steps. First, identify the security requirements that apply, from regulation, industry standards and your own policy. Second, develop the configuration itself: operating system, application and network settings. Third, test and validate it in a controlled environment, because a hardening setting that breaks an application is an outage with a security excuse. Fourth, deploy it before the system reaches production, then monitor for drift. Your sources are the Centre for Internet Security Benchmarks and the vendor hardening guides.

## Slide 7 · Configuration management tools

---

Once a baseline exists, something has to enforce it. Ansible automates provisioning, configuration and deployment using readable text files, which keeps it approachable. Puppet manages infrastructure across its lifecycle with a declarative language. Chef uses a domain specific language built on Ruby. SaltStack uses a master and minion architecture and is known for speed and scale. All four apply the approved settings, watch for drift and report a system that has left the standard, and automation is the only realistic way to hold thousands of systems to one baseline.

## Slide 8 · System hardening: shrink the attack surface

---

Hardening means shrinking the attack surface, which is every point where an attacker can send or take data. Audit the services a system is running and disable the ones it does not need, because many systems enable a wide range by default. Close the network ports nobody uses. Remove default accounts, or give any that must remain a unique password and minimal permissions. Least privilege is a practice, not a setting. Fewer entry points means less to patch, less to monitor, and less to explain after an incident.

## Slide 9 · Three layers: operating system, application, firmware

---

Hardening is applied at three levels, and exam questions like to separate them. The operating system needs regular patching, unnecessary services disabled, and a mandatory access control framework that confines each program to what it should touch. An application needs secure configuration, regular updates and input validation. Firmware needs the manufacturer updates and secure boot enabled, and secure boot refuses to load unauthorised firmware during start up. Skip one layer and the others still carry risk, because an attacker takes the easiest path available.

## Slide 10 · Balancing security and functionality

---

One caution before we leave hardening. A setting that blocks a business process will be switched off in a hurry, usually without telling the security team. So assess the impact of each change before you apply it, test it somewhere safe, document what you changed, and review it later against operational need. The goal is not the most secure machine possible in isolation, it is the most secure machine the business can actually run. Documented and tested hardening survives, while heroic hardening is rolled back.

## Slide 11 · Chapter two: mobile devices and wireless

---

Chapter two moves outside the building. A phone, a laptop and a tablet leave your control every day, join networks you do not own, and carry data you are responsible for. Wireless is a shared medium that spills past your walls, so anyone nearby can listen. We will secure the device, the data on it, and the air it speaks over. Treat a phone like the server it has become.

## Slide 12 · Mobile devices: the unique risks

---

Mobile devices fail differently. They are small, so they get lost and stolen. They run different operating systems with different security models, so one policy is hard to apply. Their app stores hand users third party applications that nobody vetted. They also roam onto public Wi-Fi, where traffic can be intercepted. Mobile device management, or MDM, answers this with central control: policy, configuration, compliance checks and remote wipe. A lost phone without a passcode is a breach in a pocket.

## Slide 13 · BYOD or corporate owned?

---

Then the question of whose device it is. Bring your own device gives flexibility and saves hardware cost, but you inherit devices that may never have met your standard. Corporate owned devices give you strict configuration control and easier compliance, at a higher cost and with less personal freedom. Either way, containerisation keeps corporate data in an isolated space that personal applications cannot reach. Application vetting then decides what may be installed, assessing each app for weak storage, poor encryption, policy compliance and developer reputation.

## Slide 14 · Wireless encryption: four generations

---

Wireless encryption has four generations, and each one exists because the previous generation failed. Wired Equivalent Privacy used the RC4 cipher with a weak integrity check, and weak keys made it breakable. Wi-Fi Protected Access added the Temporal Key Integrity Protocol, a fresh key for every packet, still on RC4. Wi-Fi Protected Access Two moved to the Advanced Encryption Standard, with a pre-shared key or enterprise mode. WPA3 introduced the Simultaneous Authentication of Equals handshake, which resists offline password guessing, and it adds forward secrecy, so a stolen key cannot unlock past sessions.

## Slide 15 · Authentication and rogue access points

---

Authentication decides who may join. A pre-shared key is one passphrase shared by everybody: simple at home, but at scale one leak opens the network. Enterprise authentication sends each credential to a RADIUS server, the Remote Authentication Dial-In User Service, so one user can be removed without changing everybody. Then there is the rogue access point, unauthorised hardware on your premises. Detect it by scanning, or with a wireless intrusion prevention system watching the spectrum. A site survey then sets coverage that stops at your walls, and it is a security control, not only a coverage exercise.

## Slide 16 · Break

---

Ten minutes. When we return: applications, assets, and the vulnerability loop.

## Slide 17 · Chapter three: applications and monitoring

---

Chapter three moves up to the application, and then to watching. Applications are where most attacks land, because they face users and they accept input. An application that accepts input is talking to a stranger. So we harden how applications are built and configured, we add defences that sit in front of them and inside them, and then we cover monitoring: what to watch, when to alert, and the trade off between how much you collect and what you can afford to keep.

## Slide 18 · Secure coding and input validation

---

Application security starts before deployment, in how the code is written. Run code with the least privilege it needs, so a compromise reaches less. Handle errors properly, because error messages leak your internals. Review code, so flaws are found before production. Then validate every input: allow only known good data, sanitise what remains, and check length and format boundaries. Allow listing beats blocklisting, because you can list the good rather than every possible bad. Most injection attacks die at a well written input check.

## Slide 19 · Application defences

---

Now the defences that sit around an application. A web application firewall filters web traffic between the application and the internet, blocking injection and cross-site scripting before they land. Sandboxing runs code in an isolated space with limited access to system resources, which contains a compromise. Allow listing means only pre-approved applications may run, which is powerful on servers that should never change. Code signing proves where software came from and that nobody altered it. And runtime application self-protection watches the running application and blocks attacks as they happen.

## Slide 20 · Discussion: prevent, contain, verify, detect

---

[Run the discussion per the instructor notes.] Here is how those pieces fit together. You will not stop every piece of malicious code from arriving, so assume something will arrive. An allow list decides what may run at

all, which stops most commodity malware before it starts. A sandbox decides what code may touch if it does run, so a compromise stays in one room rather than the building. Code signing verifies that the binary is the one the developer published. Layer them, and you cover prevent, contain, verify and detect.

## **Slide 21 · Security monitoring: what to watch**

---

Monitoring is the continuous observation of systems and networks, to spot a security event in time to act. Four things deserve your attention. Authentication attempts, because a burst of failed logins from one address is a credential attack in progress. Privilege usage, because administrator accounts are the prize, and odd activity there deserves a question. Network traffic, because a sudden outbound spike can be an exfiltration. And system changes, such as a new service or a modified configuration file. You cannot spot abnormal until you know what normal looks like.

## **Slide 22 · Alerting strategy and monitoring scope**

---

Two strategies work together. Real-time alerting catches the event as it happens, which is what you need during an active intrusion. Trend analysis reads the history instead, and that is how you notice that failed logins have been creeping up for weeks. Widen the scope and you see more of the network, but you collect far more data. That data needs storage and a retention policy that balances cost against regulation, and the more you collect, the harder you must filter, or analysts drown in alerts and miss the real one.

## **Slide 23 · Chapter four: the asset lifecycle**

---

Chapter four is asset management, and everything else rests on it. You cannot protect what you do not know you own. We follow an asset from the moment its security requirements are specified, through assignment and daily tracking, to secure destruction at end of life. Inventory, ownership, custody, licensing and disposal are unglamorous, and they also decide whether an incident stays small. You cannot patch what you cannot find.

## **Slide 24 · Asset acquisition**

---

Asset management starts before the asset arrives. When you buy hardware or software, you specify the security requirements it must meet, such as the encryption and access control features you need. You then assess the vendor, because their reputation, compliance record and security practice become your risk. You also verify the supply chain, auditing suppliers and recording where each component came from. That defends you against a supply chain attack, where a malicious change enters during manufacture or delivery. An asset chosen carelessly is risk you never scanned for.

## **Slide 25 · Assignment: owner and custodian**

---

Once an asset is in service, two roles keep it honest. The owner is the person or department accountable for the asset, and decides how it may be used. The custodian does the daily work: applying updates, controlling access, and keeping the record current. Behind both sits an accurate inventory of what exists and who holds

it, and that inventory separates an incident from a guess, because the first question after a compromise is what else touched it.

## **Slide 26 · Tracking location, configuration and patch state**

---

Keeping track is continuous, not an annual spreadsheet. Location tracking matters most for the assets that move, so a laptop that leaves the building is one you can still find and wipe. Configuration records the hardware, software, network settings and permissions on each device. Patch state tells you which systems are current, which are waiting and which are deliberately refused, so findings can be matched to the estate. Licences belong here too, along with the hunt for unauthorised installations, because an unapproved application is both a licensing risk and a security one.

## **Slide 27 · Secure disposal and data sanitisation**

---

When an asset reaches the end of its life, disposal is a security process. Overwriting replaces the data with random data, so the original cannot be recovered, which suits media that will be reused. Degaussing uses a strong magnetic field to erase magnetic storage, and does nothing for flash memory. Physical destruction, by shredding or crushing, is the answer for hardware leaving your control for good. Then verify, because a destruction process nobody checked is a hope rather than a control. Document the method and the people involved.

## **Slide 28 · Chain of custody and compliance**

---

The most overlooked asset control is the paperwork. A chain of custody records an asset life: a unique identifier, every transfer with its date, time and personnel, and the disposal with proof that the data was destroyed. That record answers an auditor, and it protects you if data from a retired drive surfaces. Regulation raises the stakes, because the General Data Protection Regulation and the Health Insurance Portability and Accountability Act impose duties on personal and health data. Disposal is audited like any other control.

## **Slide 29 · Chapter five: vulnerability management**

---

Chapter five is the heart of the session. Vulnerability management is a loop, and the loop never closes for good. Identify the weaknesses with scanners and current feeds. Analyse them against severity and context. Prioritise by risk rather than by the loudest number. Remediate, validate that the fix worked, and report honestly. A finding nobody prioritised is a decision made by default.

## **Slide 30 · Vulnerability identification: scanning tools**

---

Vulnerability management is the ongoing search for weaknesses, and it starts with scanners. Network scanners assess the infrastructure, looking at routers, switches and firewalls for open ports, insecure protocols and outdated software versions. Host scanners work on one device at a time, checking the operating system, installed applications and configuration. Application scanners are specialised, and they test

a web application for injection flaws, cross-site scripting and weak authentication. Together they find the known weaknesses on the systems you know about. None of them finds the design flaw.

## **Slide 31 · Scanning modes and coverage**

---

Two scanning modes, and the difference is examined. A credentialed scan logs in with a valid account, so it can see missing patches and misconfigured settings from the inside. A non-credentialed scan sees only what an unauthenticated attacker would see from outside. Then set your schedule against risk and regulation. Define scope carefully, because a scan that misses a critical database tells a comfortable lie. And feed the process with current intelligence, including the Common Vulnerabilities and Exposures identifiers, which name every known flaw.

## **Slide 32 · CVSS: three metric groups**

---

When a finding arrives, someone must judge it, and the standard tool is the Common Vulnerability Scoring System. It builds a score from three groups of metrics. Base metrics describe the vulnerability itself and never change: the attack vector, the attack complexity, the privileges required, and the impact on confidentiality, integrity and availability. Temporal metrics move with the world: is exploit code available, is a fix available, how confident is the report. Environmental metrics make the score yours, asking how much this system matters to your business.

## **Slide 33 · Beyond the score: deciding priority**

---

A score is not a decision. Three contextual factors decide priority on top of the number. Asset criticality asks whether the system runs a critical function or holds crown jewel data. Exposure asks whether the asset is reachable, and an internet facing service with a medium score usually matters more than an isolated machine with a high one. Exploitability asks whether working exploit code exists, and whether attackers are already using it. Then validate, because a false positive consumes scarce effort. Exposure and active exploitation beat the raw score.

## **Slide 34 · Activity: prioritise and validate**

---

[Run the activity per the instructor notes. 6 minutes in pairs, 4 minutes debrief.] An internet facing service with a medium score and exploit code in the wild: remediate now. An isolated test server with a critical score and no exploit: later in the queue. A finding the engineer cannot reproduce: validate before acting. Which factors outweigh the score? Exposure and active exploitation.

## **Slide 35 · Remediation options**

---

Now the response, and there are four options. Patching updates the software and removes the flaw, once a vendor has released a fix. A configuration change can close the door instead, by disabling an unnecessary service or tightening a default setting. A compensating control covers what you cannot fix yet: extra monitoring, enhanced logging, or an intrusion detection system watching that asset, which reduces the risk

without removing the flaw. Risk acceptance is a documented decision that fixing costs more than the impact, and it is reviewed regularly. Every exception needs an owner, a risk statement and a date.

## **Slide 36 · Validation, exceptions and reporting**

---

Do not assume the fix worked. Rescanning confirms that the patch applied, the configuration change took effect and the compensating control is working. Where a vulnerability cannot be fixed yet, the exception process keeps it visible: document the reason, assess the risk, add a temporary control, set a date and review it. Then report: counts by severity, average time to patch and the percentage of systems patched, then read the trends. Give executives impact and engineers detail. Two more duties belong here. Disclosure should be responsible, with the vendor given time to fix before the public knows, and a penetration test runs only under written rules of engagement, because testing without permission is the crime.

## **Slide 37 · Chapter six: monitoring tools and technologies**

---

Chapter six brings the tooling together. Every control we have discussed produces logs, and logs on their own change nothing. We need platforms that collect, correlate and store the evidence, and people who can read it. Aggregation, correlation, real time alerting, behaviour analytics, dashboards and alert tuning are the final piece. See the event, understand it, and act before the damage spreads. A tool nobody tunes is an expensive log file.

## **Slide 38 · Monitoring tooling**

---

A security information and event management system collects logs from firewalls, detection systems and servers, then correlates them, so an event that looks harmless in one log becomes obvious in three. A log management platform is the foundation: collection, storage and analysis, with retention that satisfies investigations and regulators. Network traffic analysis captures and inspects packets, catching exfiltration or a flood in progress. And file integrity monitoring watches critical files against a known baseline, which is how you spot tampering and attacker persistence.

## **Slide 39 · Behaviour analytics, dashboards and alert tuning**

---

Two more capabilities, and one discipline. User and entity behaviour analytics learn what normal looks like for each account and each device, then flag the deviation, which catches a compromised account that no signature would recognise. It also risk scores its findings, so investigations start where the behaviour is strangest. Then the discipline: a dashboard should carry a few critical metrics, drawn clearly, so the trend is visible at a glance, and alerts need thresholds and rules refined until false positives stop crowding out the real ones.

## Slide 40 · Quiz

---

[Direct candidates to [Quiz\\_Session\\_10.html](#). 15 minutes.] Twenty questions, immediate feedback, and every question carries the session area it tests. Read the explanations, because the distractors teach as much as the answers. Aim for seventy percent or better. The Wayground version is available for in-class assessment.

## Slide 41 · Key takeaways

---

Seven takeaways. Build from a baseline, and remember that drift is the enemy. Harden three layers, which means fewer services, ports and accounts. Secure the edge: current encryption and central control. Defend the application by validating input, allow listing, sandboxing and signing. Account for every asset from acquisition to certified destruction. Run the loop: scan, score, remediate, rescan and report. And tune your alerts, or your analysts will miss the real one.

## Slide 42 · Revision checklist

---

Before next session: retake the quiz until you score seventy percent without guessing. Pick one system and write the baseline settings you would enforce. Walk a room and count the assets, naming the owner and the patch state of each. Find one flaw you cannot patch, and write its compensating control and review date. And read ahead to Session 11: security operations, monitoring and incident response.

## Slide 43 · Next session preview

---

That closes Session Ten. You can now build from a hardened baseline, secure the devices and the airwaves, defend the application, and account for every asset from purchase to destruction. You can find vulnerabilities, judge them by context, fix what matters first, and report what remains, and you can instrument the estate so the evidence reaches you in time to act. Next time, Security Operations: monitoring, incident response and digital forensics, where we take the evidence these tools produce and use it. I will see you there.